

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ
Факультет физико-математических и естественных наук

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 4

Студент: Николаева Ангелина Борисовна

Студенческий билет: 1032253612

Группа: НКАбд-04-25

МОСКВА

2026 г

Содержание

1. Цель работы	3
2. Выполнение лабораторной работы	4
2.1.Создание программы Hello world!	4
2.2.Работа с транслятором NASM	5
2.3.Работа с расширенным синтаксисом командной строки NASM.....	5
2.4.Работа с компоновщиком LD.....	5
2.5.Запуск исполняемого файла	5
2.6.Задание для самостоятельной работы	6
2.7.Контрольные вопросы для самопроверки	7
3.Вывод	9

1. Цель работы.

Цель данной лабораторной работы - освоить процедуры компиляции и сборки программ, написанных на ассемблере NASM.

2. Выполнение лабораторной работы

2.1. Создание программы Hello world!

В домашней директории создаю каталог, в котором буду хранить файлы для текущей лабораторной работы.
(рис.1)



```
abnikilaeva@fedora:~$ mkdir -p ~/work/arch-pc/lab04
abnikilaeva@fedora:~$ cd ~/work/arch-pc/lab04
abnikilaeva@fedora:~/work/arch-pc/lab04$
```

Рисунок 1

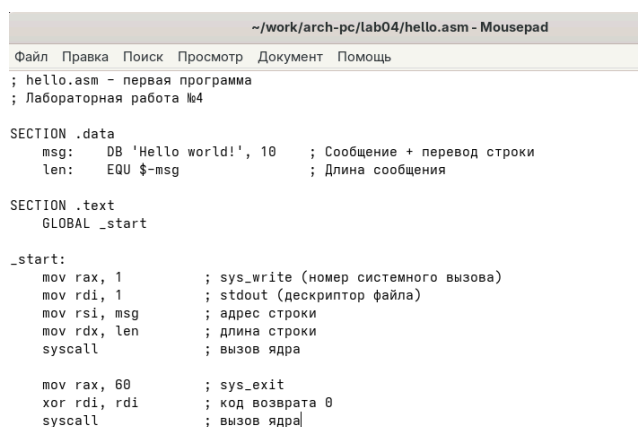
Создаю в нем файл hello.asm, в котором буду писать программу на языке ассемблера. (рис.2)



```
abnikilaeva@fedora:~$ mkdir -p ~/work/arch-pc/lab04
abnikilaeva@fedora:~$ cd ~/work/arch-pc/lab04
abnikilaeva@fedora:~/work/arch-pc/lab04$ touch hello.asm
abnikilaeva@fedora:~/work/arch-pc/lab04$ mousepad hello.asm
abnikilaeva@fedora:~/work/arch-pc/lab04$
```

Рисунок 2

С помощью редактора пишу программу в созданном файле. (рис. 3)



```
~/work/arch-pc/lab04/hello.asm - Mousepad
Файл  Правка  Поиск  Просмотр  Документ  Помощь
; hello.asm - первая программа
; Лабораторная работа №4

SECTION .data
    msg:    DB 'Hello world!', 10    ; Сообщение + перевод строки
    len:    EQU $-msg                ; Длина сообщения

SECTION .text
    GLOBAL _start

_start:
    mov rax, 1        ; sys_write (номер системного вызова)
    mov rdi, 1        ; stdout (дескриптор файла)
    mov rsi, msg       ; адрес строки
    mov rdx, len       ; длина строки
    syscall            ; вызов ядра

    mov rax, 60        ; sys_exit
    xor rdi, rdi       ; код возврата 0
    syscall            ; вызов ядра
```

Рисунок 3

2.2. Работа с транслятором NASM.

Компилирую с помощью NASM свою программу. (рис.4)

```
abnikilaeva@fedora:~/work/arch-pc/lab04$ nasm -f elf64 hello.asm -o hello.o
abnikilaeva@fedora:~/work/arch-pc/lab04$ ls
hello.asm  hello.o
abnikilaeva@fedora:~/work/arch-pc/lab04$
```

Рисунок 4

2.3. Работа с расширенным синтаксисом командной строки NASM.

Выполняю команду, указанную на (рис. 5), она скомпилировала исходный файл hello.asm в obj.o, расширение .o говорит о том, что файл - объектный, помимо него флаги -g -l подготовят файл отладки и листинга соответственно.

```
abnikilaeva@fedora:~/work/arch-pc/lab04$ nasm -f elf64 hello.asm -o hello.o
abnikilaeva@fedora:~/work/arch-pc/lab04$ ls
hello.asm  hello.o
abnikilaeva@fedora:~/work/arch-pc/lab04$ nasm -o obj.o -f elf64 -g -l list.lst hello.asm
abnikilaeva@fedora:~/work/arch-pc/lab04$ ls
hello.asm  hello.o  list.lst  obj.o
abnikilaeva@fedora:~/work/arch-pc/lab04$
```

Рисунок 5

2.4. Работа с компоновщиком LD.

Затем мне необходимо передать объектный файл компоновщику, делаю это с помощью команды ld. (рис.6)

```
abnikilaeva@fedora:~/work/arch-pc/lab04$ ld -m elf_i386 hello.o -o hello
abnikilaeva@fedora:~/work/arch-pc/lab04$ ls
hello  hello.asm  hello.o  list.lst  obj.o
abnikilaeva@fedora:~/work/arch-pc/lab04$ ld -m elf_i386 obj.o -o main
abnikilaeva@fedora:~/work/arch-pc/lab04$ ls
hello  hello.asm  hello.o  list.lst  main  obj.o
abnikilaeva@fedora:~/work/arch-pc/lab04$
```

Рисунок 6

Выполняю следующую команду ..., результатом исполнения команды будет созданный файл main, скомпонованный из объектного файла obj.o. (рис.6)

2.5. Запуск исполняемого файла.

Запускаю исполняемый файл из текущего каталога. (рис.7)

```
abnikilaeva@fedora:~/work/arch-pc/lab04$ ./hello
Hello, world!
abnikilaeva@fedora:~/work/arch-pc/lab04$
```

Рисунок 7

2.6. Задание для самостоятельной работы.

Создаю копию файла для последующей работы с ней. (рис.8)

```
abnikilaeva@fedora:~/work/arch-pc/lab04$ cp hello.asm lab4.asm
abnikilaeva@fedora:~/work/arch-pc/lab04$ ls
hello  hello.asm  hello.o  lab4.asm  list.lst  main  obj.o
abnikilaeva@fedora:~/work/arch-pc/lab04$
```

Рисунок 8

Редактирую копию файла, заменив текст на свое имя и фамилию. Транслирую копию файла в объектный файл, компоную и запускаю. (рис.9)

```
abnikilaeva@fedora:~/work/arch-pc/lab04$ ls
hello  hello.asm  hello.o  lab4  lab4.asm  lab4.o  list.lst  main  obj.o
abnikilaeva@fedora:~/work/arch-pc/lab04$ ./lab4
Nikolaeva Angelina
abnikilaeva@fedora:~/work/arch-pc/lab04$
```

Рисунок 9

Убедившись в корректности работы программы, копирую рабочие файлы в свой локальный репозиторий. (рис. 10)

```
abnikilaeva@fedora:~/work/arch-pc/lab04$ cp hello.asm lab4.asm ../../study/2024-2025/arch/arch-pc/labs/lab04/
abnikilaeva@fedora:~/work/arch-pc/lab04$ cd ../../study/2024-2025/arch/arch-pc/labs/lab04/
abnikilaeva@fedora:~/work/study/2024-2025/arch/arch-pc/labs/lab04$ ls
hello.asm  lab4.asm  presentation  report  texput.log
abnikilaeva@fedora:~/work/study/2024-2025/arch/arch-pc/labs/lab04$
```

Рисунок 10

Загрузка изменений на свой удаленный репозиторий на GitHub. (рис.11)

```
abnikilaeva@fedora:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/lab04$ git add .
abnikilaeva@fedora:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/lab04$ git status
On branch master
Your branch is up to date with 'origin/master'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   hello.asm
    new file:   lab4.asm

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   ../lab03/report/report.docx
    modified:   ../lab03/report/report.pdf

.....
abnikilaeva@fedora:~/work/arch-pc/lab04$ cp hello.asm lab4.asm ../../study/2024-2025/arch/arch-pc/labs/lab04/
abnikilaeva@fedora:~/work/arch-pc/lab04$ cd ../../study/2024-2025/arch/arch-pc/labs/lab04/
abnikilaeva@fedora:~/work/study/2024-2025/arch/arch-pc/labs/lab04$ ls
hello.asm  lab4.asm  presentation  report  texput.log
abnikilaeva@fedora:~/work/study/2024-2025/arch/arch-pc/labs/lab04$
```

Рисунок 11

2.7. Контрольные вопросы для самопроверки.

1. Какие основные отличия ассемблерных программ от программ на языках высокого уровня?

- Близость к аппаратуре: Ассемблер — машинно-ориентированный язык низкого уровня, который позволяет получить более полный доступ к ресурсам компьютера по сравнению с языками высокого уровня (C/C++, Python и др.).
- Детальность управления: Ассемблерная программа содержит только тот код, который ввёл программист, без скрытых действий со стороны компилятора (оптимизаций, добавления служебных проверок и т.п.).
- Одна команда — одна строка: В ассемблерной программе каждая команда располагается на отдельной строке (размещение нескольких команд на одной строке недопустимо), тогда как в языках высокого уровня это возможно.
- Чувствительность к регистру: Синтаксис NASM чувствителен к регистру (разница между большими и малыми буквами), что не всегда свойственно языкам высокого уровня.
- Прямое использование регистров: В ассемблере операндами команд часто являются регистры процессора, а доступ к ним осуществляется по именам, а не по адресам, как к памяти.

2. В чём состоит отличие инструкции от директивы на языке ассемблера?

- Инструкция (мнемокод) — это непосредственно команда процессору, которая переводится в машинный код.
- Директива — это инструкция для транслятора (ассемблера), которая не переводится в машинные команды, а управляет процессом трансляции. Директивы используются для определения данных, резервирования памяти, задания секций и т.д.

3. Перечислите основные правила оформления программ на языке ассемблера.

- Каждая команда располагается на отдельной строке.
- Формат записи команды: [метка:] мнемокод [операнд {, операнд}] [; комментарий].
- Метка (если есть) связывается с адресом данной команды или данных.
- Допустимые символы в метках: буквы, цифры \$, #, @, -, _, ?. Метка может начинаться с буквы, точки (.), _ или ?.
- Максимальная длина идентификатора — 4095 символов.
- Для экранирования зарезервированных слов используется символ \$ перед идентификатором.
- Директивы обычно пишутся большими буквами (но это традиция, а не строгое правило).
- Регистр букв важен (чувствительность к регистру).

4. Каковы этапы получения исполняемого файла?

Четыре основных этапа (согласно схеме на рис. 4.3):

1. Набор текста программы в текстовом редакторе и сохранение в файл с расширением .asm.
2. Трансляция — преобразование текста программы в машинный код (объектный

файл) с помощью транслятора (например, `nasm`). Расширение объектного файла — `.o`.

3. Компоновка (линковка) — обработка объектного кода компоновщиком (`ld`), который собирает исполняемый файл (обычно без расширения).

4. Запуск программы (и при необходимости — отладка).

5. Каково назначение этапа трансляции?

Трансляция преобразует текст программы на языке ассемблера (с мнемониками) в машинный код (последовательности битов), который понимает процессор.

Результатом трансляции является объектный файл (`.o`). Также на этом этапе может быть создан файл листинга (`.lst`), содержащий дополнительную информацию от транслятора.

6. Каково назначение этапа компоновки?

Компоновка (линковка) принимает на вход один или несколько объектных файлов, связывает их, разрешает внешние ссылки (например, на метки `_start`) и формирует готовый исполняемый файл, который можно запускать. Компоновщик также может создавать файл карты загрузки (`.map`).

7. Какие файлы могут создаваться при трансляции программы, какие из них создаются по умолчанию?

- Файлы, которые могут создаваться:
 - `.o` — объектный файл (по умолчанию при использовании `nasm -f elf hello.asm` создаётся `hello.o`).
 - `.lst` — файл листинга (создаётся по ключу `-l`).
 - исполняемый файл (без расширения) — создаётся компоновщиком, не транслятором.
- Создаётся по умолчанию: объектный файл с именем исходного файла, но с расширением `.o` (если не указано иное ключом `-O`).

8. Каковы форматы файлов для `nasm` и `ld`?

- NASM (транслятор): формат объектного файла задаётся ключом `-f`. Для Linux используются:
 - `elf` — для 32-битных версий.
 - `elf64` — для 64-битных версий.
- LD (компоновщик): на вход принимает объектные файлы формата ELF. Для 32-битных программ используется ключ `-m elf_i386`. Результат — исполняемый файл в формате ELF (для Linux).

3.Вывод

При выполнении данной лабораторной работы я освоила процедуры компиляции и сборки программ, написанных на ассемблере NASM.